
Project Title: Script-Controlled ACL – Restrict Record Access Based on Field Value

Date: 28 September 2026

Team ID: SWTID-2026-2244

PHASE 3 – PROJECT DESIGN

1. Introduction:

The Project Design phase describes the structure and workflow of the proposed ServiceNow access-control system.

The design focuses on controlling access to **Institution Details** records using user roles, ACLs, scripts, and the Branch field.

2. System Architecture:

The project follows the following access-control structure:

User → Role → ACL → Script/Data Condition → Record Access

The user attempts to access a record. ServiceNow checks the user's assigned role and the corresponding ACL. For the Read operation, the Branch datacondition is also checked.

ccc

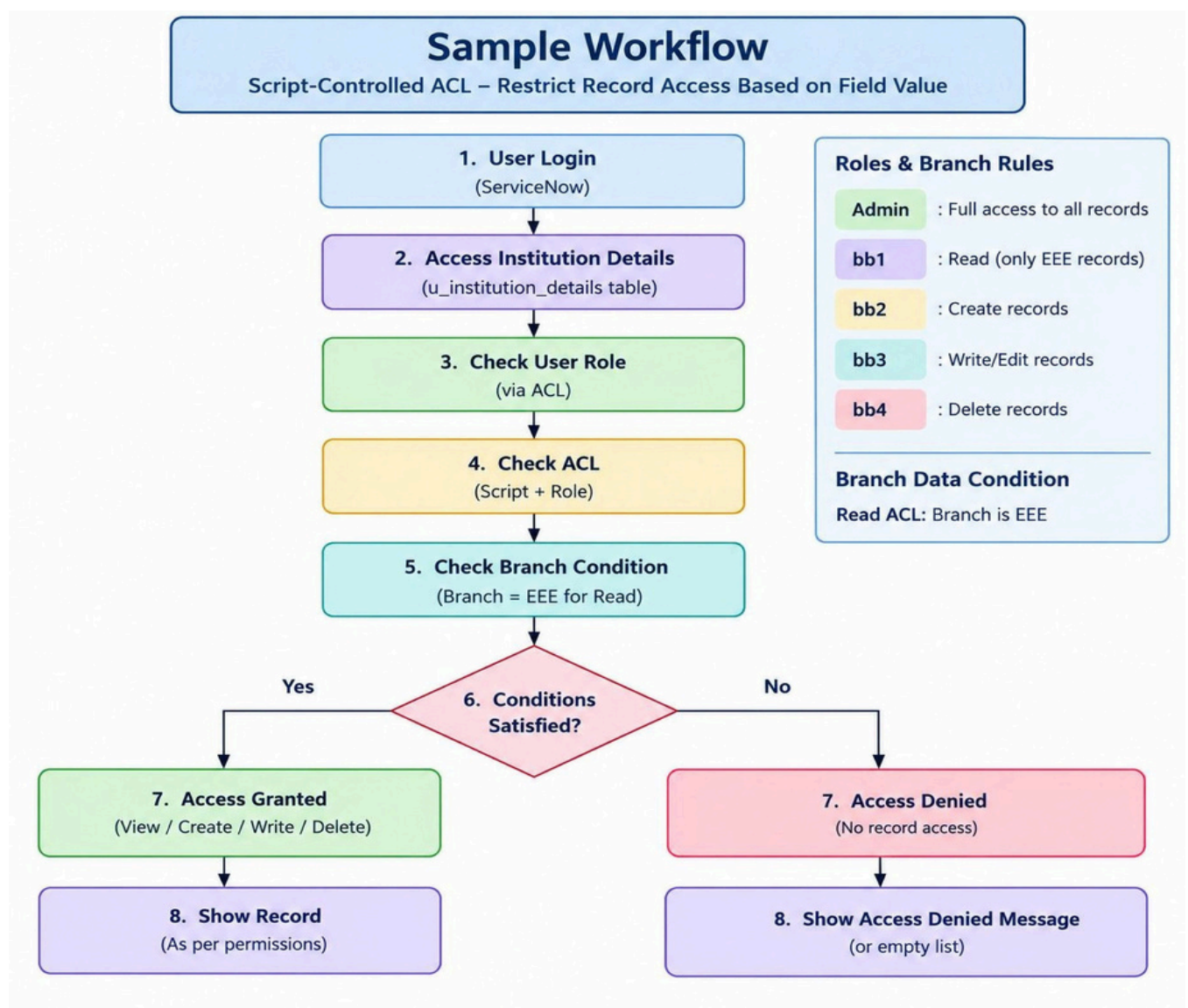
3. System Workflow:

The basic workflow is:

1. User logs in to ServiceNow.

2. User attempts to access the **Institution Details** table.
3. ServiceNow checks the applicable ACL.
4. The user's assigned role is verified.
5. For Read access, the Branch condition is checked.
6. If the required conditions are satisfied, access is granted.
7. If the conditions are not satisfied, access is denied.
8. Administrators receive full access

Sample Workflow Diagram:



4. Table Design:

The custom table used in the project is:

Label: Institution Details

Table Name: u_institution_details

Table Fields

Field	Field Type	Purpose
Student Roll Number	Auto Number	Unique student identification
Student Name	Reference – User	Stores student information
Faculty Name	Reference – User	Stores faculty information
Branch	Choice	Stores EEE, ECE, or CSE
Email	String	Stores email address
Phone Number	String	Stores phone number
Description	Multi String	Stores additional information

5. Branch Design:

The **Branch** field is designed as a Choice field.

The available choices are:

- **EEE**
- **ECE**
- **CSE**

For the ReadACL, the data condition is configured as:

Branch is EEE

Therefore, the branch value acts as a record-level restriction for the intended EEE access scenario.

6. Role Design:

The project uses four custom roles:

Role	Operation
bb1	Read
bb2	Create
bb3	Write
bb4	Delete

The **admin** role is given full access through the ACL script.

7. ACL Design:

Four record-level ACLs are planned:

Read ACL:

- Table: u_institution_details
- Operation: Read
- Role: bb1
- Advanced: Yes
- Data Condition: Branch is EEE
- Script: Checks whether the user has admin or bb1.

Create ACL:

- Table: u_institution_details
- Operation: Create
- Role: bb2

Write ACL:

- Table: u_institution_details
- Operation: Write
- Role: bb3

Delete ACL:

- Table: u_institution_details
- Operation: Delete
- Role: bb4

8. Access-Control Logic:

The Read ACL uses the following logic:

```
(function () {  
  if(gs.hasRole('admin')) {  
    return true;  
  }  
  
  if(gs.hasRole('bb1')) {  
    return true;  
  }  
  
  return false;  
})();
```

The **script checks the user's role**, while the **Data Condition Branch is EEE restricts the applicable records to EEE records**.

s9.AccessMatrix:

User/Role	EEE Records	ECE Records	CSE Records
Admin Allowed	bb1 user	Allowed	Allowed
Allowed No required role		Restricted	Restricted
Denied		Denied	Denied

For Create, Write, and Delete operations, the corresponding bb2, bb3, and bb4 roles are required.

10. Security Design:

The system is designed to prevent unauthorized access by using multiple security layers:

- User authentication
- Role-based permissions
- ACL rules
- Script-based validation
- Branch-based data condition
- Separate permissions for Create, Read, Write, and Delete operations

This design ensures that users receive only the access required for their assigned responsibilities.

11. Expected Design Outcome:

The final design should provide:

- Controlled access to Institution Details.
- Branch-based record restriction.
- Role-based CRUD permissions.

- Full administrator access.
 - Prevention of unauthorized record operations.
 - A structured and maintainable ACL configuration.
-

12. Conclusion:

The Project Design phase defines the architecture, table structure, roles, ACLs, workflow, and security logic of the system. This design will be used as the blueprint for implementing the project in ServiceNow.
